

Transition to turbulence under high enthalpy conditions

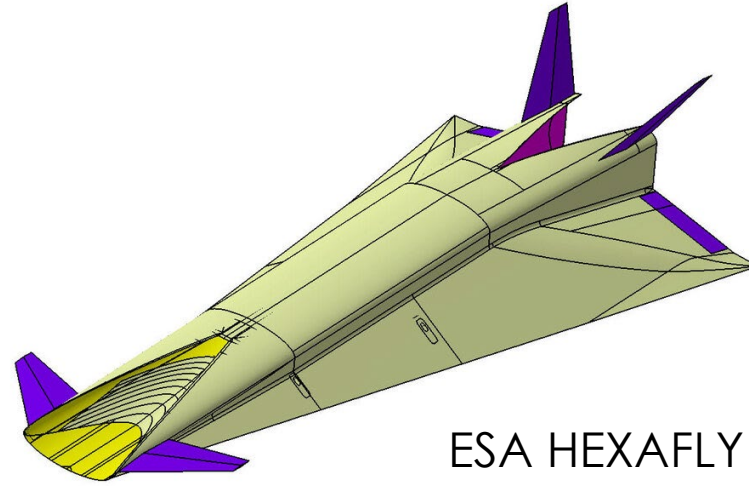
Neil Sandham, Teja Ala, Ali Musawi and Max Walker

Aeronautics and Astronautics,
University of Southampton

Hyperstructures 2026, Nottingham

Motivation: vehicle design

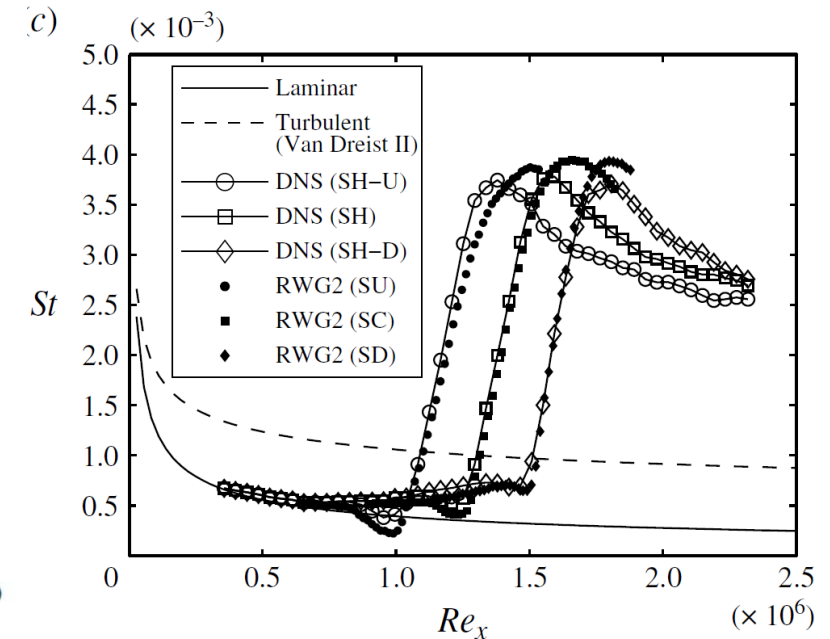
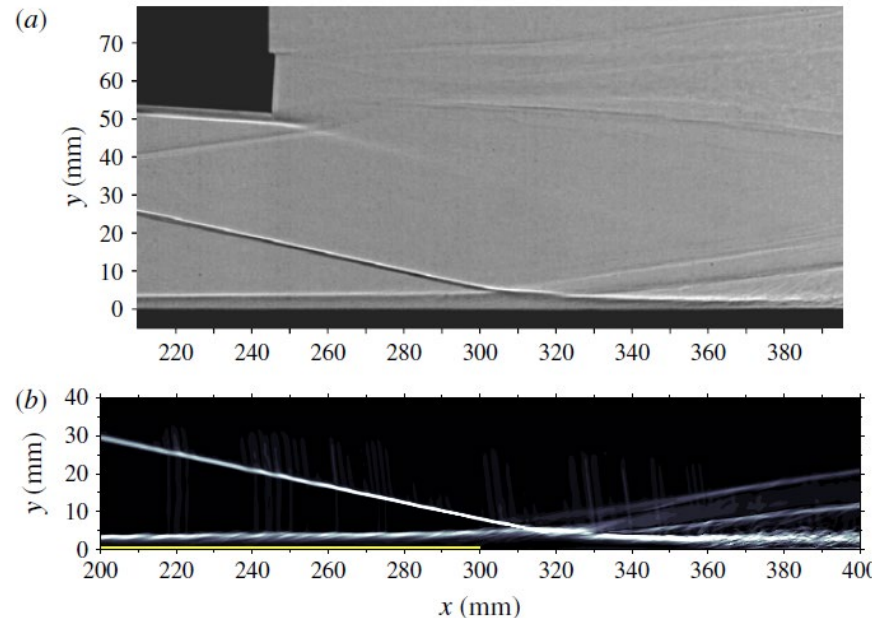
- NASP (1988): “Estimates [of transition] range from 20% to 80% along the body ...The estimate made for the point of transition can affect the design vehicle gross take-off weight by a factor of two or more.”
- Jacobs & Steelant (2025): reductions up to 22% in GTOW and 31% in fuel consumption if a high-speed slender Mach 8 cruise vehicle would be designed accounting for transitional flow rather than the conservative fully turbulent flow



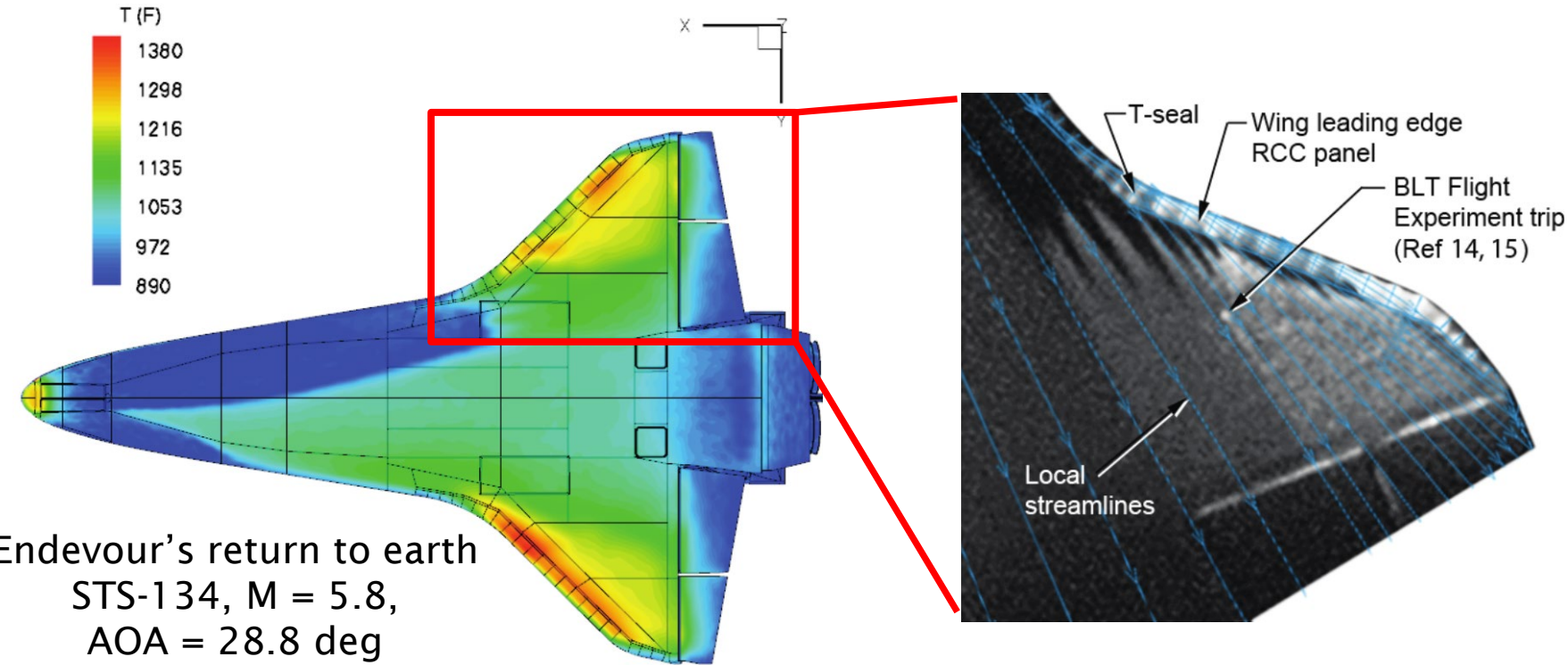
ESA HEXAFLY vehicle
concept for Mach 8 flight

- Excessive heating when transition is combined with shock-wave/boundary-layer interaction

$M=6$ transitional SBLI
expt vs. DNS (JFM 752)



Effect of surface protrusions



Horvath et al, *Global Infrared Observations of Roughness Induced Transition on the Space Shuttle Orbiter*, RTO-MP-AVT-200

Discrete roughness (vs. distributed roughness, Reda)

Outline

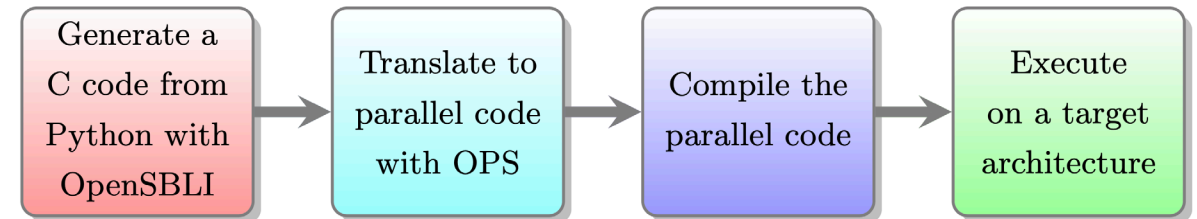
- Code base & numerical aspects
 - OpenSBLI/OPS code-generation framework
 - High enthalpy continuum formulation
 - Efficient property (μ , κ) evaluation
- Hypersonic applications: nonequilibrium effects in mixing and transition to turbulence

OpenSBLI Python-based code-generation

High-level problem specification via symbolic algebra, linked to OPS (Mudalige, Reguly)
- originated in EPSRC software future-proofing project (Jammy et al, 2016)

- OpenSBLI is a **Python-based** code-generation framework for compressible CFD using finite-difference schemes [1,2].
- Users **specify the equations to solve** and simulation options in Python.
- **Symbolic algebra (SymPy)** is converted into a complete ILES/DNS CFD solver in C/C++ code.
- The OPS DSL enables **parallel execution on many platforms** (MPI/OpenMP/CUDA/...).
- In use at several institutes for high-fidelity simulations of compressible CFD (including NASA, JAXA).

```
1 ndim = 3
2 sc1 = "**{'scheme':'Teno'}"
3 # Define the compressible Navier-Stokes equations in Einstein notation.
4 mass = "Eq(Der(rho,t), - Conservative(rhou_j,x_j,%s))" % sc1
5 momentum = "Eq(Der(rhou_i,t), -Conservative(rhou_i*u_j + KD(_i,_j)*p,x_j , %s) + Der(tau_i_j,x_j) )" % sc1
6 energy = "Eq(Der(rhoE,t), - Conservative((p+rhoE)*u_j,x_j, %s) - Der(q_j,x_j) + Der(u_i*tau_i_j ,x_j) )" % sc1
7 stress_tensor = "Eq(tau_i_j, (mu/Re)*(Der(u_i,x_j)+ Der(u_j,x_i) - (2/3)* KD(_i,_j)* Der(u_k,x_k)))"
8 heat_flux = "Eq(q_j, (-mu/((gama-1)*Minf*Minf*Pr*Re))*Der(T,x_j))"
9 # Numerical scheme selection
10 Avg = RoeAverage([0, 1])
11 LLF = LLFTeno(teno_order, averaging=Avg)
12 cent = Central(4)
13 rk = RungeKuttaLS(3, formulation='SSP')
14 # Specifying boundary conditions
15 boundaries[direction][side] = IsothermalWallBC(direction, 0, wall_eqns)
16 # Generate a C code
17 alg = TraditionalAlgorithmRK(block)
18 OPSC(alg)
```



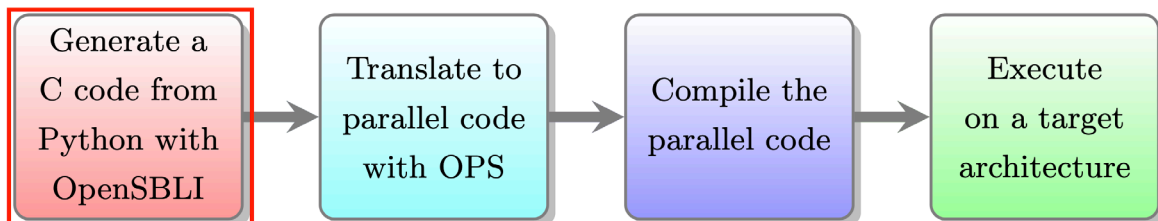
[1] D.J. Lusher, S.P. Jammy, N.D. Sandham. OpenSBLI: Automated code-generation for heterogeneous computing architectures applied to compressible fluid dynamics on structured grids. **Computer Physics Communications** (2021).

[2] D.J. Lusher, et al. OpenSBLI v3.0: High-fidelity multi-block transonic aerofoil CFD simulations using domain specific languages on GPUs . **Computer Physics Communications** (2025).

OpenSBLI workflow: I

Specify the physical problem and numerical parameters in Python:

- Define **one set of equations** in index notation (SimulationEquations).
- Expanded over the number of dimensions selected.
- Placeholder variable formulae** (pressure - p, temperature - T, ...) can be defined separately.
- Heat sinks, source-terms, forcing** etc, can be added at the Python level.
- Select numerical schemes (central-WENO-TENO-hybrid), boundary conditions, initial conditions, precision, I/O options.



```

1 ndim = 3
2 sc1 = "**{'scheme':'Teno'}"
3 # Define the compressible Navier-Stokes equations in Einstein notation.
4 mass = "Eq(Der(rho,t), - Conservative(rhou_j,x_j,%s))" % sc1
5 momentum = "Eq(Der(rhou_i,t), -Conservative(rhou_i*u_j + KD(_i,_j)*p,x_j , %s) + Der(tau_i_j,x_j) )" % sc1
6 energy = "Eq(Der(rhoE,t), - Conservative((p+rhoE)*u_j,x_j , %s) - Der(q_j,x_j) + Der(u_i*tau_i_j,x_j) )" % sc1
7 stress_tensor = "Eq(tau_i_j, (mu/Re)*(Der(u_i,x_j)+ Der(u_j,x_i) - (2/3)* KD(_i,_j)* Der(u_k,x_k)))"
8 heat_flux = "Eq(q_j, (-mu/((gama-1)*Minf*Minf*Pr*Re))*Der(T,x_j))"
9 # Numerical scheme selection
10 Avg = RoeAverage([0, 1])
11 LLF = LLFTeno(teno_order, averaging=Avg)
12 cent = Central(4)
13 rk = RungeKuttaLS(3, formulation='SSP')
14 # Specifying boundary conditions
15 boundaries[direction][side] = IsothermalWallBC(direction, 0, wall_eqns)
16 # Generate a C code
17 alg = TraditionalAlgorithmRK(block)
18 OPSC(alg)
  
```

Defining the system of PDEs to solve as Python strings

$$\begin{aligned}
 \frac{\partial}{\partial t}(\rho) &= - \frac{\rho \cdot \frac{\partial}{\partial x_0}(u_0)}{2} - \frac{u_0 \cdot \frac{\partial}{\partial x_0}(\rho)}{2} - \frac{\frac{\partial}{\partial x_0}(\rho u_0)}{2} \\
 \left[\begin{aligned}
 \frac{\partial}{\partial t} \left(\rho \cdot \frac{\partial}{\partial x_0} \left(\frac{u_0^2}{2} \right) + 2 \cdot \rho u_0 \cdot \frac{\partial}{\partial x_0}(u_0) + u_0 \cdot \frac{\partial}{\partial x_0}(\rho) + 2 \cdot u_0 \cdot \frac{\partial}{\partial x_0}(\rho u_0) + \frac{\partial}{\partial x_0}(\rho u_0 \cdot u_0) \right) \\
 \frac{\partial}{\partial t}(\rho u_0) &= - \frac{\frac{\partial}{\partial x_0}(\rho u_0)}{4} - \frac{\frac{\partial}{\partial x_0}(p)}{4} \\
 \frac{\partial}{\partial t}(\rho u_0) &= - \frac{H \cdot \rho \cdot \frac{\partial}{\partial x_0}(u_0)}{4} - \frac{H \cdot u_0 \cdot \frac{\partial}{\partial x_0}(\rho)}{4} - \frac{H \cdot \frac{\partial}{\partial x_0}(\rho u_0)}{4} - \frac{\rho \cdot \frac{\partial}{\partial x_0}(H \cdot u_0)}{4} - \frac{\rho u_0 \cdot \frac{\partial}{\partial x_0}(H)}{4} - \frac{u_0 \cdot \frac{\partial}{\partial x_0}(H \cdot \rho)}{4} - \frac{\frac{\partial}{\partial x_0}(H \cdot \rho u_0)}{4}
 \end{aligned} \right]
 \end{aligned}$$

Symbolic pretty print representation of a cubic-split form in SymPy

$$wk3_B0[0] = inv_0 \cdot \left(- \frac{2 \cdot H_B0[-1]}{3} + \frac{H_B0[-2]}{12} + \frac{2 \cdot H_B0[1]}{3} - \frac{H_B0[2]}{12} \right)$$

Symbolic representation of a finite-difference derivative approximation

OpenSBLI workflow: II

Translate the code using the OPS DSL:

- OpenSBLI writes a (**sequential**) code with calls to the OPS library API.
- Defines all storage arrays, constants, access stencils, loops to evaluate and I/O calls.
- OPS Python translator generates MPI, OpenMP, CUDA, OpenCL... versions of the code.

```
int range64[] = {-3, b0np0 + 4, -3, b0np1 + 4, -3, b0np2 + 4};
ops_par_loop(opensbliblock00Kernel064,
"Constituent Relations evaluation", opensbliblock00, 3, range64,
ops_arg_dat(Et_B0, 1, stencil_0_00, "double", OPS_READ),
ops_arg_dat(rho_B0, 1, stencil_0_00, "double", OPS_READ),
ops_arg_dat(u0_B0, 1, stencil_0_00, "double", OPS_READ),
ops_arg_dat(u1_B0, 1, stencil_0_00, "double", OPS_READ),
ops_arg_dat(u2_B0, 1, stencil_0_00, "double", OPS_READ),
ops_arg_dat(a_B0, 1, stencil_0_00, "double", OPS_WRITE),
ops_arg_dat(p_B0, 1, stencil_0_00, "double", OPS_RW));
```

```
void opensbliblock00Kernel064(const ACC<double> &Et_B0, const ACC<double> &rho_B0,
const ACC<double> &u0_B0, const ACC<double> &u1_B0, const ACC<double> &u2_B0,
ACC<double> &a_B0, ACC<double> &p_B0)
{
    double inv_rho = 0.0;
    inv_rho = 1.0/rho_B0(0,0,0);

    p_B0(0,0,0) = (gamma - 1)*(Et_B0(0,0,0) - 0.5*pow(u0_B0(0,0,0), 2)
    - 0.5*pow(u1_B0(0,0,0), 2) - 0.5*pow(u2_B0(0,0,0), 2))*rho_B0(0,0,0);

    a_B0(0,0,0) = sqrt(gama*inv_rho*p_B0(0,0,0));
}
```

Sequential



Source-to-source translation step

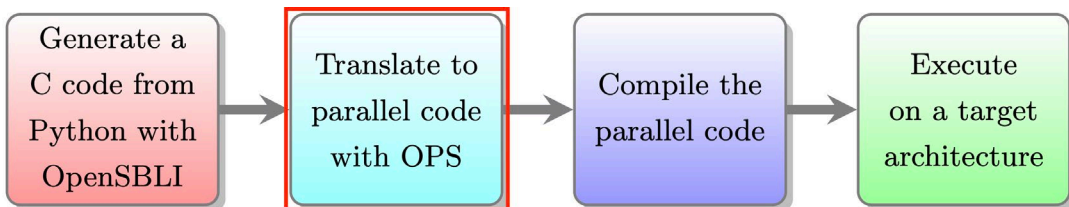
Parallel versions

```
__global__ void ops_opensbliblock00Kernel064(
double* __restrict arg0,
double* __restrict arg1,
double* __restrict arg2,
double* __restrict arg3,
double* __restrict arg4,
double* __restrict arg5,
double* __restrict arg6,
int size0,
int size1,
int size2){

    int idx_z = blockIdx.z * blockDim.z + threadIdx.z;
    int idx_y = blockIdx.y * blockDim.y + threadIdx.y;
    int idx_x = blockIdx.x * blockDim.x + threadIdx.x;

    arg0 += idx_x * 1*1 + idx_y * 1*1 * dims_opensbliblock00Kernel064[0][0] + idx_z * 1*1 * dims_opensbliblock00Kernel064[0][0] * dims_opensbliblock00Kernel064[0][1];
    arg1 += idx_x * 1*1 + idx_y * 1*1 * dims_opensbliblock00Kernel064[1][0] + idx_z * 1*1 * dims_opensbliblock00Kernel064[1][0] * dims_opensbliblock00Kernel064[1][1];
    arg2 += idx_x * 1*1 + idx_y * 1*1 * dims_opensbliblock00Kernel064[2][0] + idx_z * 1*1 * dims_opensbliblock00Kernel064[2][0] * dims_opensbliblock00Kernel064[2][1];
    arg3 += idx_x * 1*1 + idx_y * 1*1 * dims_opensbliblock00Kernel064[3][0] + idx_z * 1*1 * dims_opensbliblock00Kernel064[3][0] * dims_opensbliblock00Kernel064[3][1];
    arg4 += idx_x * 1*1 + idx_y * 1*1 * dims_opensbliblock00Kernel064[4][0] + idx_z * 1*1 * dims_opensbliblock00Kernel064[4][0] * dims_opensbliblock00Kernel064[4][1];
    arg5 += idx_x * 1*1 + idx_y * 1*1 * dims_opensbliblock00Kernel064[5][0] + idx_z * 1*1 * dims_opensbliblock00Kernel064[5][0] * dims_opensbliblock00Kernel064[5][1];
    arg6 += idx_x * 1*1 + idx_y * 1*1 * dims_opensbliblock00Kernel064[6][0] + idx_z * 1*1 * dims_opensbliblock00Kernel064[6][0] * dims_opensbliblock00Kernel064[6][1];

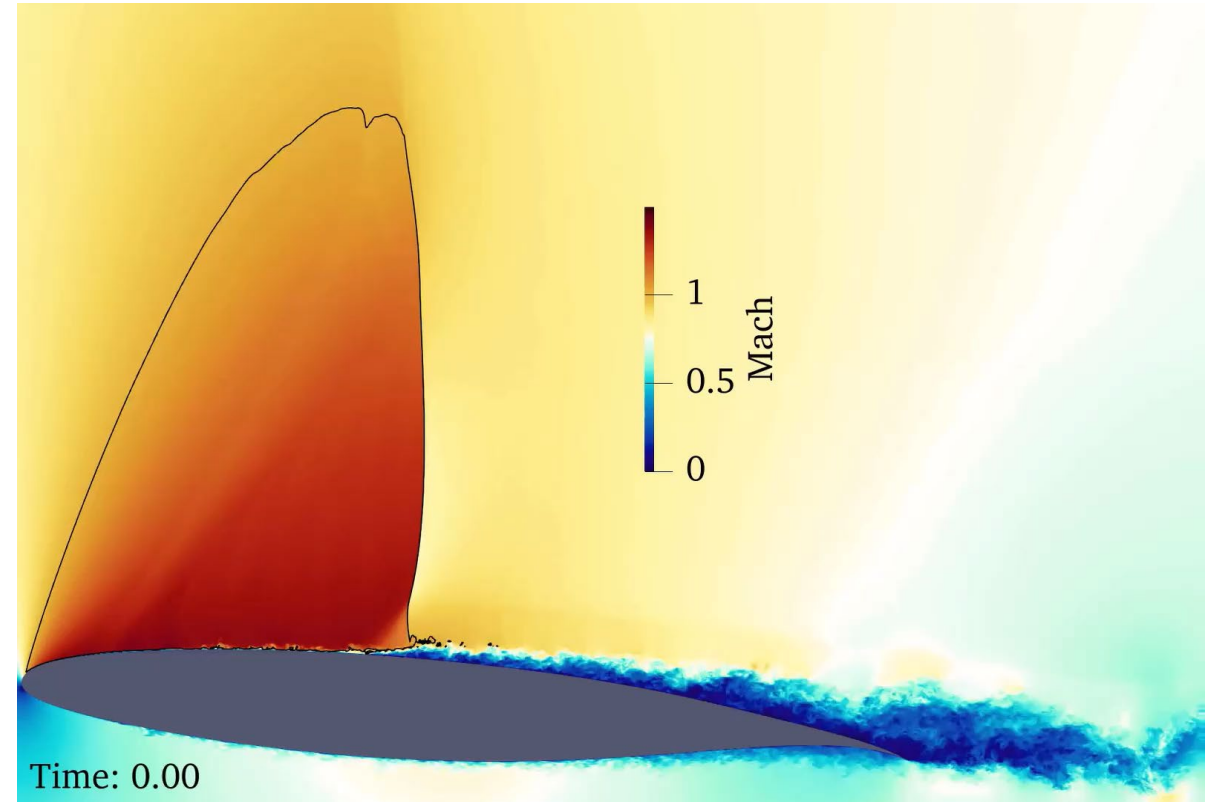
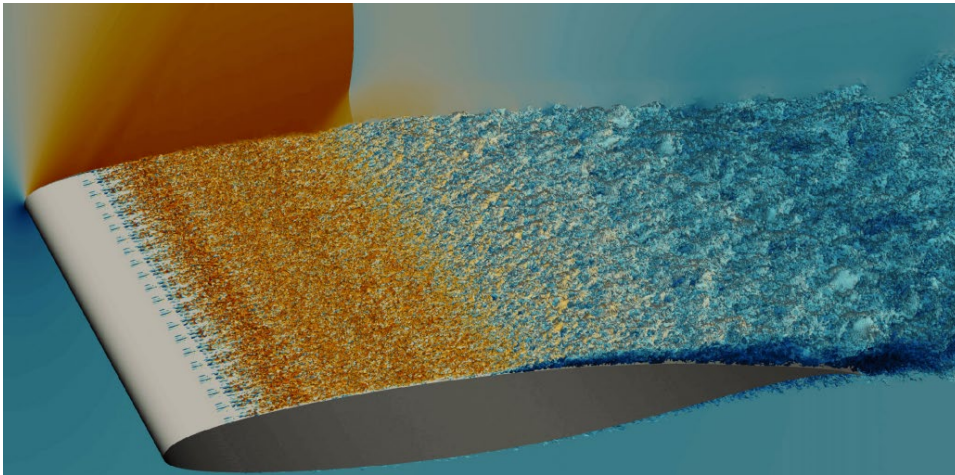
    if (idx_x < size0 && idx_y < size1 && idx_z < size2){
        const ACC<double> argp0(dims_opensbliblock00Kernel064[0][0], dims_opensbliblock00Kernel064[0][1], arg0);
        const ACC<double> argp1(dims_opensbliblock00Kernel064[1][0], dims_opensbliblock00Kernel064[1][1], arg1);
        const ACC<double> argp2(dims_opensbliblock00Kernel064[2][0], dims_opensbliblock00Kernel064[2][1], arg2);
        const ACC<double> argp3(dims_opensbliblock00Kernel064[3][0], dims_opensbliblock00Kernel064[3][1], arg3);
        const ACC<double> argp4(dims_opensbliblock00Kernel064[4][0], dims_opensbliblock00Kernel064[4][1], arg4);
        const ACC<double> argp5(dims_opensbliblock00Kernel064[5][0], dims_opensbliblock00Kernel064[5][1], arg5);
        const ACC<double> argp6(dims_opensbliblock00Kernel064[6][0], dims_opensbliblock00Kernel064[6][1], arg6);
        opensbliblock00Kernel064_gpu(argp0, argp1, argp2, argp3,
        argp4, argp5, argp6);
    }
}
```



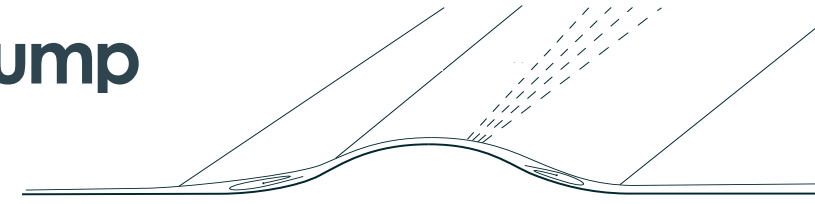
Transonic application: airfoil buffet

(Lusher et al, JAXA, AIAAJ 2024, JFM 2025)

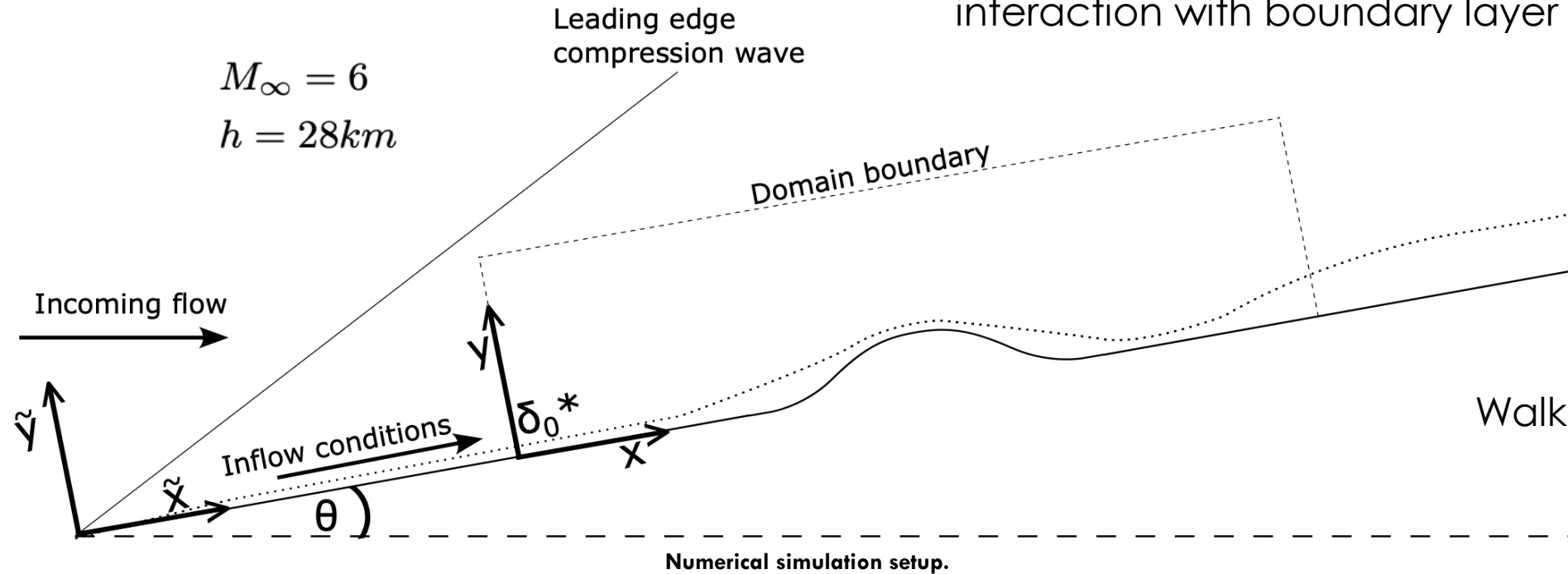
- NASA CRM airfoil section
- $M=0.72$, $Re=500,000$, AoA 5 deg
- Tripped
- Up to 7.5 billion grid points (AR=3 case) on 120 V100 GPU computer



Supersonic application: Gaussian bump



Multiple separations; shock and expansion interaction with boundary layer



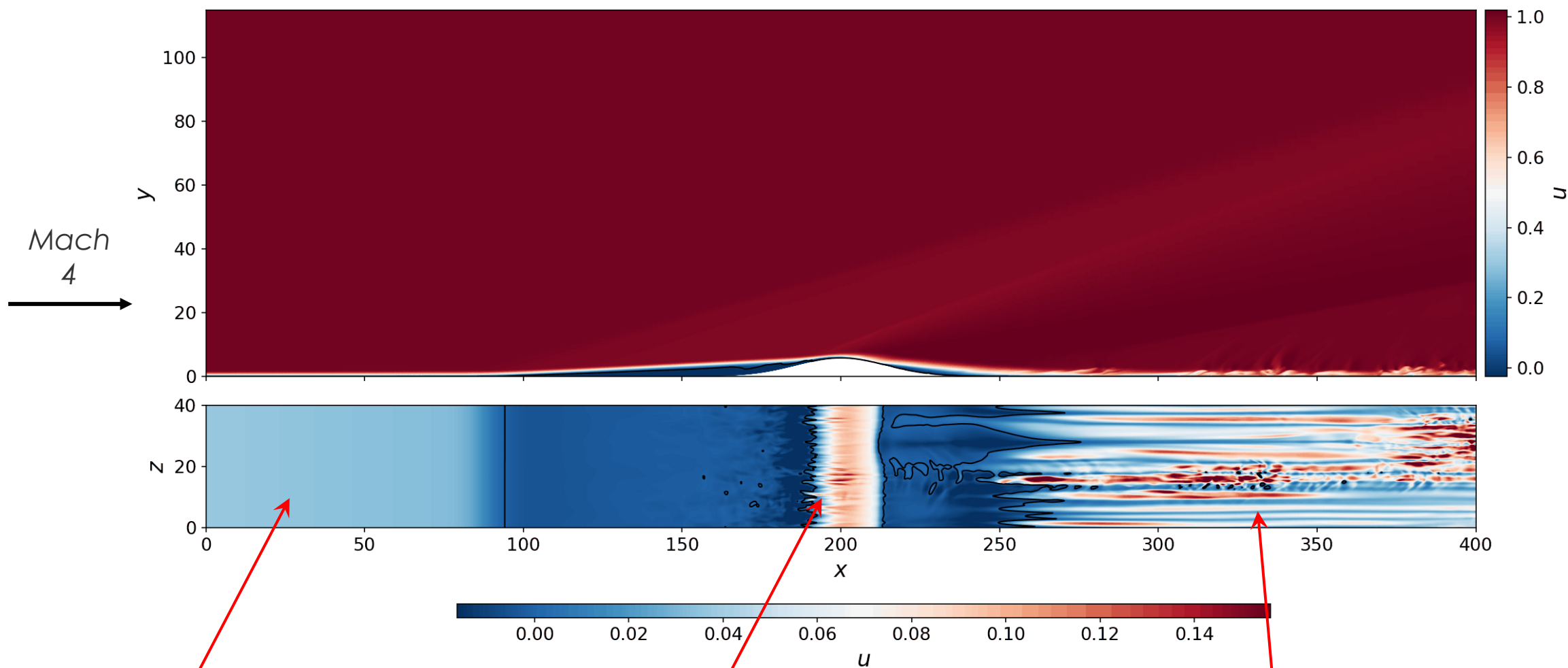
Walker (AIAA J, 2026)

Wedge-based inflow conditions

Case	wedge angle (θ)	M_∞	P_∞ (KPa)	T_∞ (K)	ρ_∞ (kg/m ³)	T_w/T_∞	H_0 (MJ/kgK)
1	15.0	4.0	9.82	439	0.078	1.37	1.85
2	7.38	5.0	4.38	307	0.049	1.95	1.85

Gaussian bump

Midspan (top) and near wall wall normal (bottom) views of streamwise velocity to show the intermittent transition

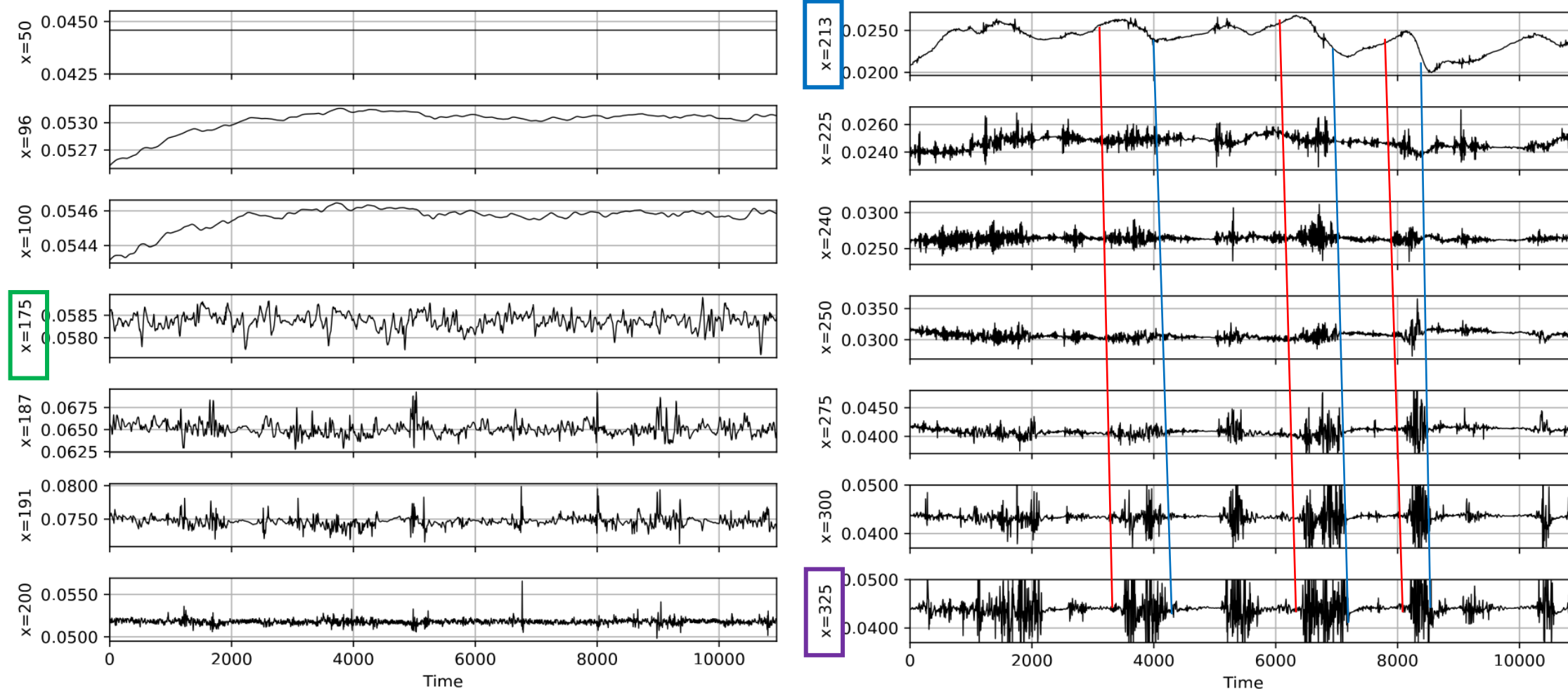


No upstream forcing

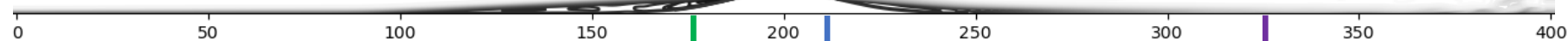
Oscillator (unstable global mode of upstream bubble)

Intermittent (spotty) transition (low-f response of downstream bubble)

Transient probe data (wall pressure)



Mid-span wall pressure at multiple streamwise locations (shown on the left of each subplot)



Governing Equations for High-Enthalpy Flows

CONSERVATION OF:

Mass

$$\frac{\partial}{\partial t} \rho_s + \frac{\partial}{\partial x_j} \rho_s u_j = \frac{\partial}{\partial x_j} \left(\rho D_s \frac{\partial X_s}{\partial x_j} \right) + \dot{\omega}_s$$

Time rate of change

Flux terms

Reactions/diffusion

Momentum

$$\frac{\partial}{\partial t} \rho u_i + \frac{\partial}{\partial x_j} \rho u_i u_j = - \frac{\partial p}{\partial x_j} + \frac{\partial}{\partial x_j} \left[\mu \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} \right) - \frac{2}{3} \mu \frac{\partial u_k}{\partial x_k} \delta_{ij} \right]$$

Pressure

Viscous

Energy

$$\frac{\partial}{\partial t} \rho E + \frac{\partial}{\partial x_j} \rho H u_j = \frac{\partial}{\partial x_j} \left(\kappa_{tr} \frac{\partial T}{\partial x_j} + \kappa_v \frac{\partial T_v}{\partial x_j} \right) + \frac{\partial}{\partial x_j} \left[u_i \mu \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} \right) - \frac{2}{3} u_i \mu \frac{\partial u_k}{\partial x_k} \delta_{ij} \right]$$

$$+ \frac{\partial}{\partial x_j} \left(\rho \sum_s h_s D_s \frac{\partial X_s}{\partial x_j} \right)$$

Thermal Conduction

Energy Exchange

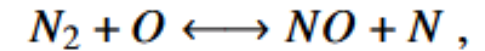
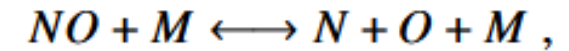
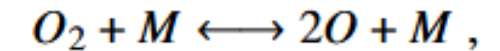
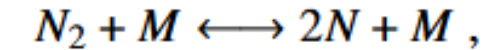
Vibrational Energy

$$\frac{\partial}{\partial t} \rho e_{v,m} + \frac{\partial}{\partial x_j} \rho e_{v,m} u_j = \frac{\partial}{\partial x_j} \left(\kappa_{v,m} \frac{\partial T_v}{\partial x_j} \right) + \rho_m \frac{(e_{v,m}^* - e_{v,m})}{\tau_m} + \frac{\partial}{\partial x_j} \left(\rho e_{v,m} D_m \frac{\partial X_m}{\partial x_j} \right) + \dot{\omega}_m \widehat{D}_m$$

Modelling for hypersonic flow

Chemical Model:

- Multi temperature 5-species air finite rate processes
- Coefficients from Park (1993) & Park et al (2001)
- Binary diffusion model by Curtiss and Hirschfelder (1949)



Thermal Model:

- Rigid Rotor Harmoni Oscillator (RRHO) model
- Landau-Teller formulation
- Relaxation in form of Millikan-White with coefficients from Park (1993)
- One transport equation per molecule or combined into one equation

Property models (viscosity, thermal conductivity):

- Blottner, Yos-Gupta
- Limited range of applicability and expensive in DNS/LES context

Slip flow regime:

- Knudsen slip boundary condition
- Earlier work on porous surface simulation (Wartemann et al AIAA J 2012)

Simplified property model

(avoids computationally expensive mixing rules)

For μ , K and K_{tr} :

$$\Phi = \frac{\sum_{i=atom} 15X_i + \sum_{i=mol} 30X_i}{P(a, b, c, d) \sum_{i=atom} X_i + P(e, f, g, h) \sum_{i=mol} X_i},$$

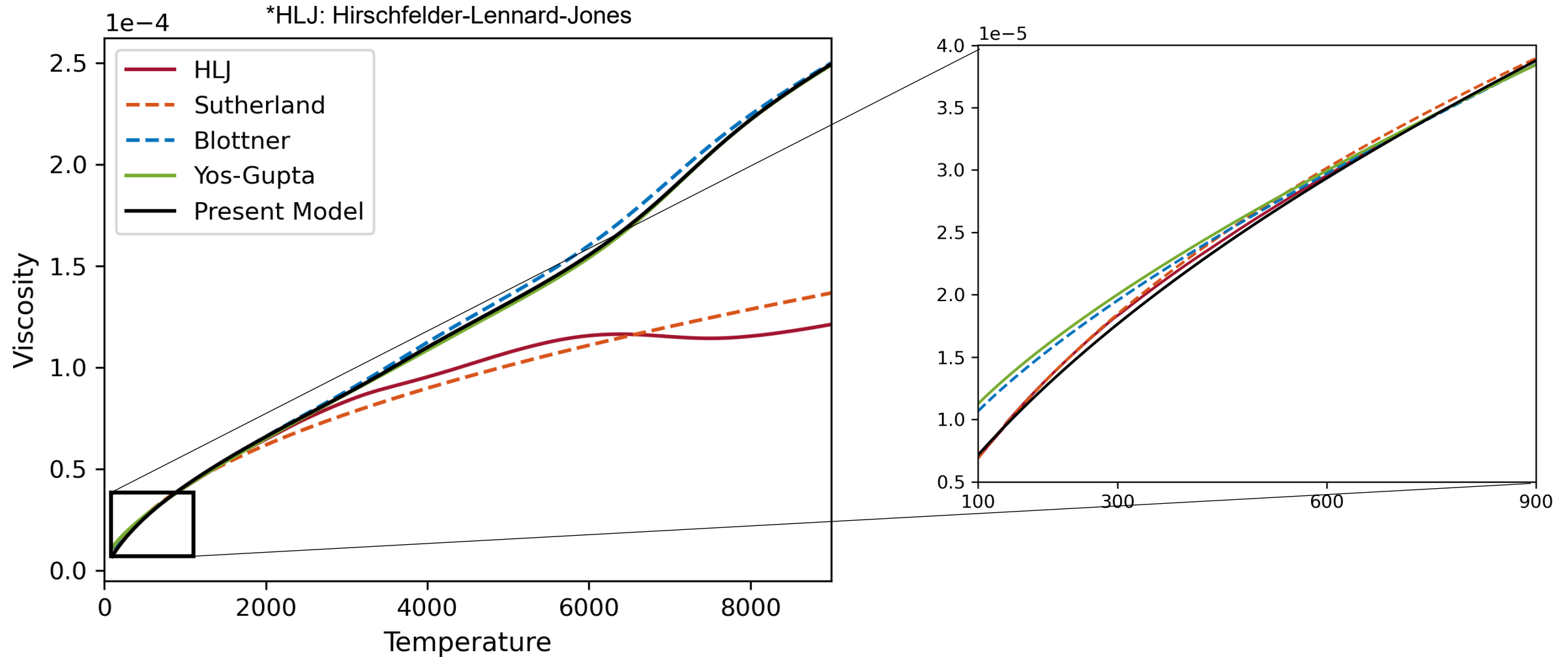
where $P(A, B, C, D) = |A + B T^2 + C \ln(T) + D/T|$,

For K_v :

$$K_{vi} = |X_i \exp(A_i + B_i T_d + C_i T_d^2 + D_i T_d^3 + E_i T_d^4 + F_i T_d^5 + G_i T_d^6)|, \quad K_v = \sum_{i=mol} K_{vi}.$$

Optimised for Hirschfelder-Lennard-Jones at low temperatures and Yos-Gupta at high temperatures
Range of applicability: 100 – 9000 K

Test of simplified viscosity model for equilibrium air



(Musawi & Sandham AIAA SciTech 2025)

2D Mixing Layer

Initial Conditions:

Fluid State:

$$M_c = 0.4$$

$$Re = 400$$

$$u = 450 \text{ m/s}$$

$$\rho = 0.02 \text{ kg/m}^3$$

$$T = 3100 \text{ K}$$

$$\delta_\omega = 0.003702 \text{ m}$$

Mixture composition:

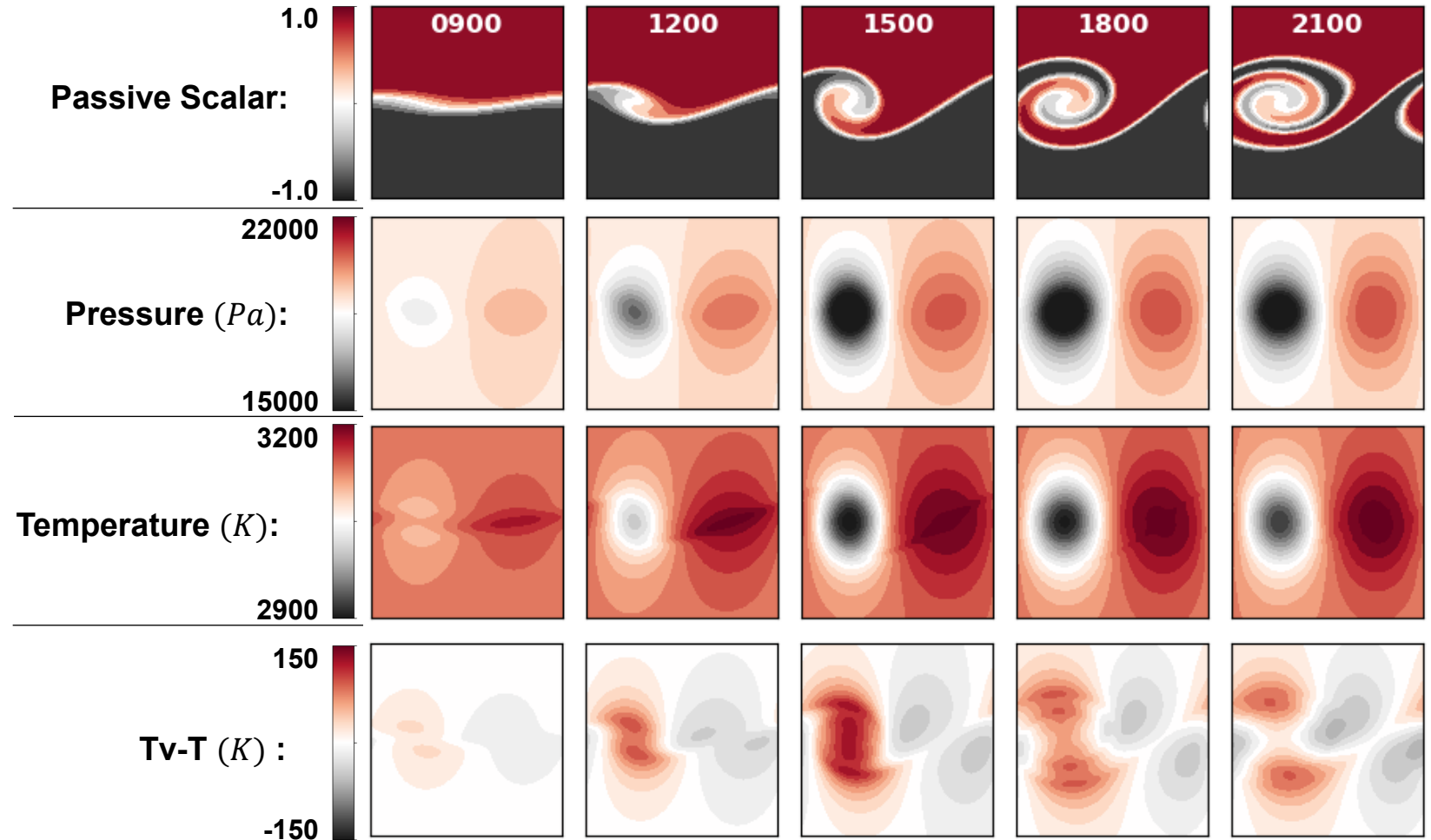
| O: 06%

| O₂: 15%

| N: 00%

| N₂: 75%

| NO: 04%



Vortex roll up with thermal non-equilibrium.

Viscosity error vs Yos-Gupta <1% vs 4% for Blottner

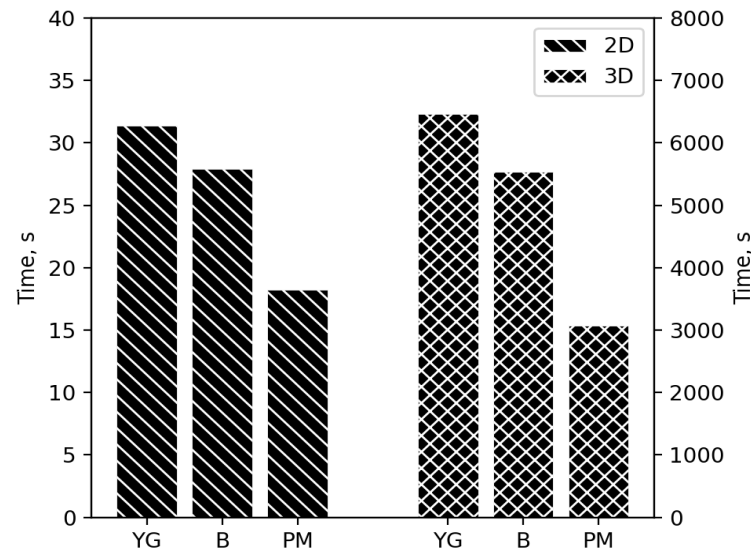
Computational Performance

Assessment:

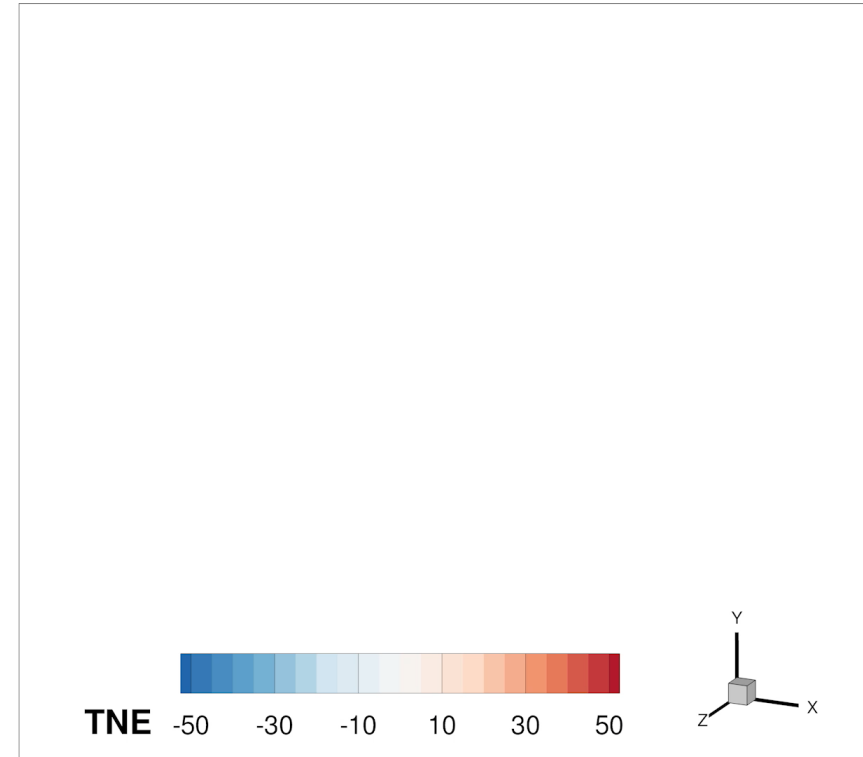
- Single node, using a single NVIDIA **A100** GPU
- Comparing run time (excluding input/output)

Note: Using OpenSBLI's work array storage reduction with no shock capturing scheme.

Grid resolution: 2D = 180x325
3D = 220x385x220



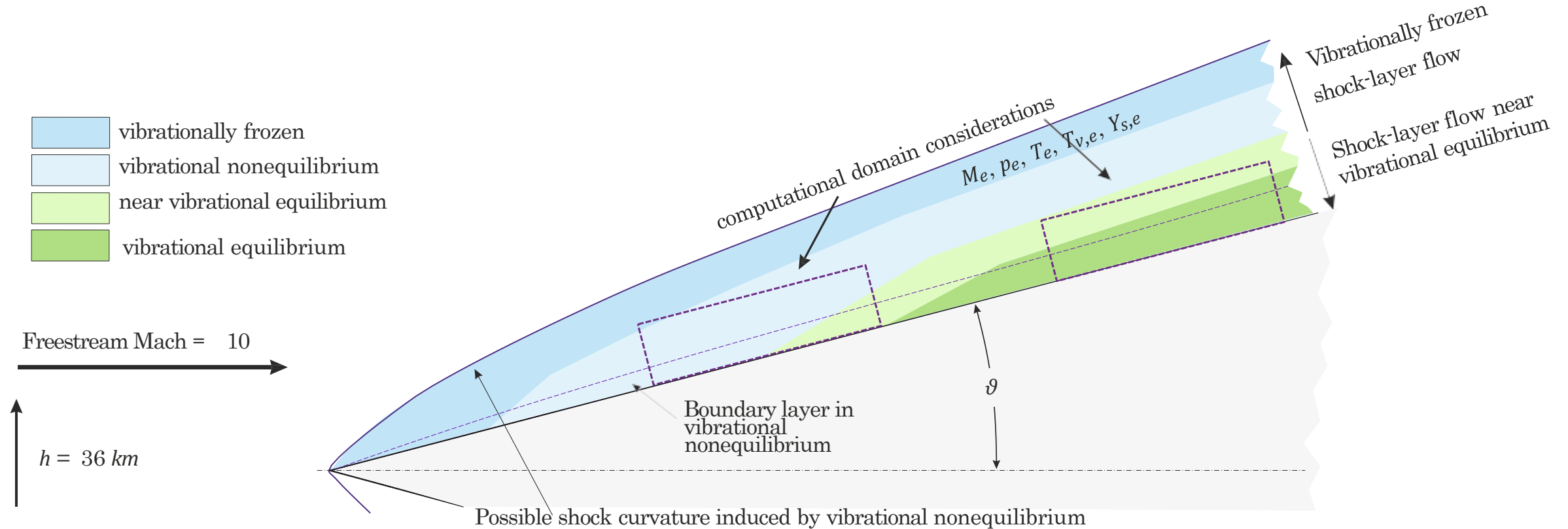
Computational performance of mixing layer cases. YG: Yos-Gupta, B: Blottner and PM: Present Model.



3D mixing layer: Q-Criterion iso-surface, coloured by $T_v - T$.

Momentum thickness error 0.5% vs Yos-Gupta
(Compared to 1.5% for Blottner)

Test case: hypersonic flow over a wedge



Schematic of the wedge adapted from [Williams et al. \(2021\)](#) and flight conditions extrapolated from [Rizvi et al. \(2015\)](#) following optimal trajectory of a hypersonic glide vehicle at its heat rate limit.

Williams et. al. (2021) *Annual Research Briefs, Stanford University*, 2021(2):119-128.

Rizvi et. al. (2015) *Defence Technology*, 11(4):350-361.

ϑ	M_e	T_e (K)	p_e (kPa)	ρ_e (kg/m ³)
12.41	6	620.0	4.91	2.748×10^{-3}
5.88	8	365.0	1.78	1.6920×10^{-3}

Direct Numerical Simulations (DNS)

- OpenSBLI framework: 4th-order CD along with RK-3 for temporal marching

Linear Stability Theory (LST)

- *Base flows*: similarity solutions for CPG, TEQ and TFG
- *Perturbations*: CPG ([Wartemann et al., 2019](#))

Calorically perfect gas (CPG)

- No vibrational energy contribution
- Variable viscosity
- Constant Prandtl number and ratio of specific heats

Thermally Frozen (TFG)

- Energy modes are no longer defined with a single temperature
- Thermal relaxation assumed to be infinitesimally slow

Thermal Equilibrium (TEQ)

- Energy modes can be described with a single temperature, i.e. $T = T_v$
- Translational-rotational and vibrational modes considered

Thermal nonequilibrium (TNE)

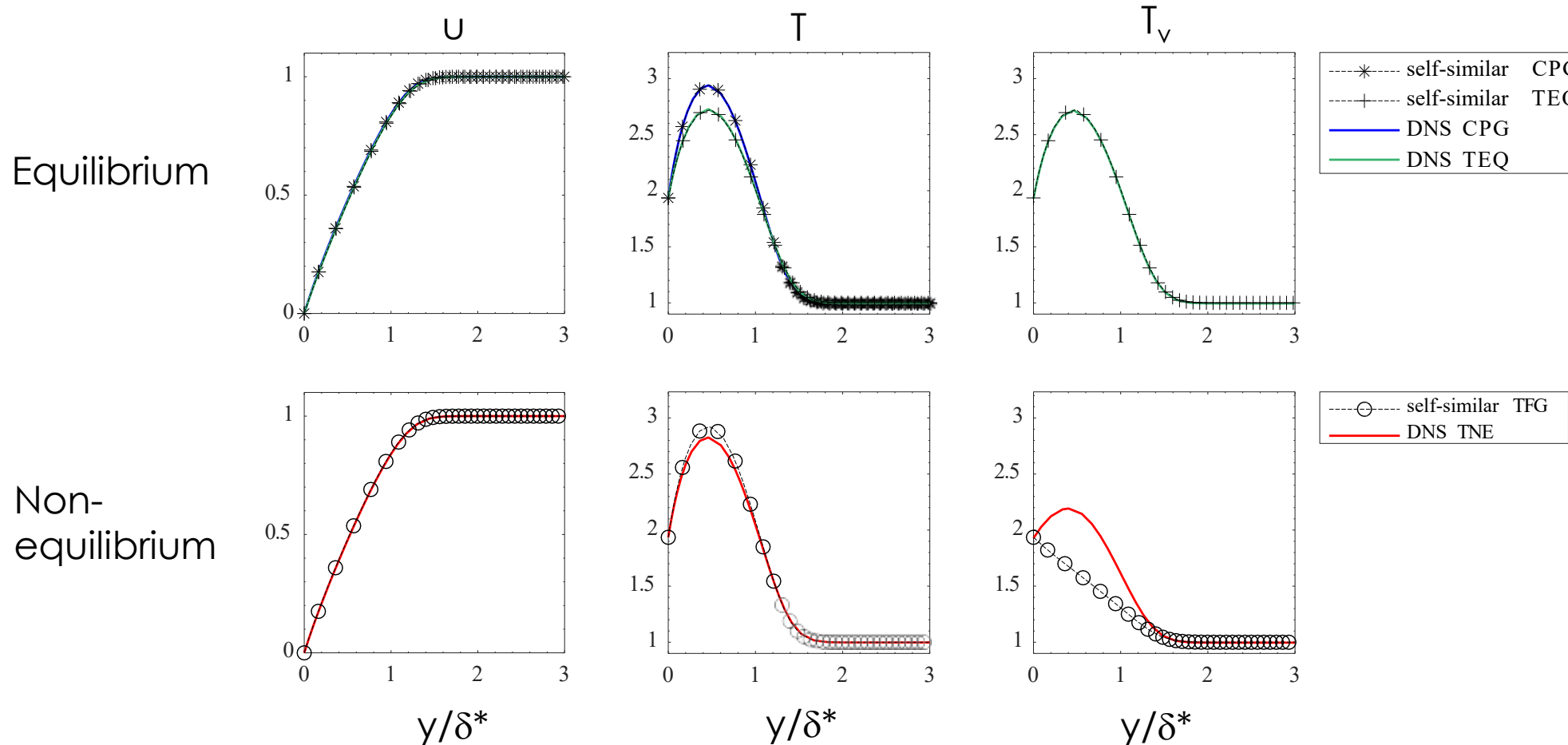
- Thermal relaxation is finite and modelled using RRHO with constants from [Park \(1993\)](#)
- Corresponding transport properties

Wartemann et. al. (2019) *Journal of Spacecraft and Rockets*, 50(2):347-356

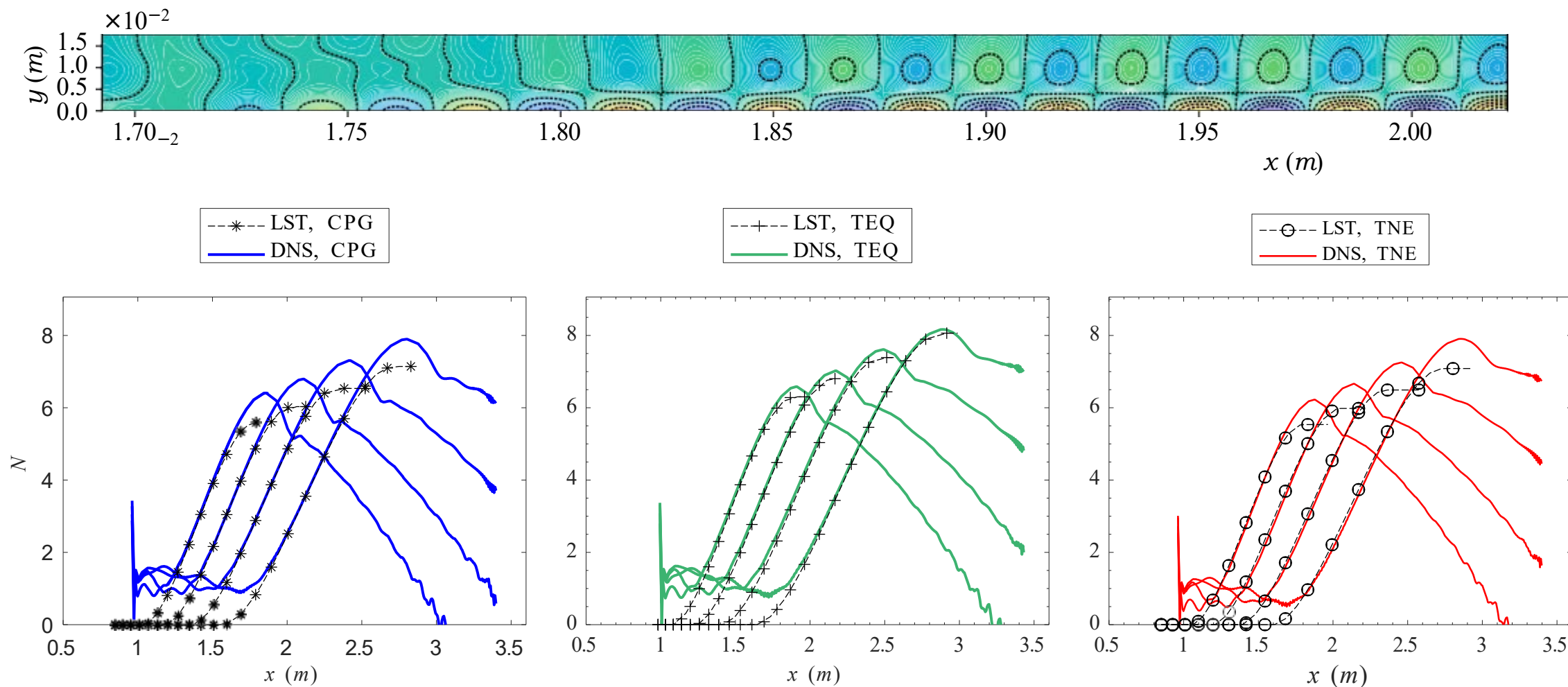
Park C. (1993) *Journal of Thermophysics and Heat Transfer*, 7(3):285-398

Validation of similarity solutions against DNS (cold wall)

M_e	T_e	T_w	Y_{N_2}	Y_{O_2}	Re_x
6	620 K	1200 K	0.76	0.24	$2.34 \times 10^6 m^{-1}$



N-factors, Mach 6 cold wall

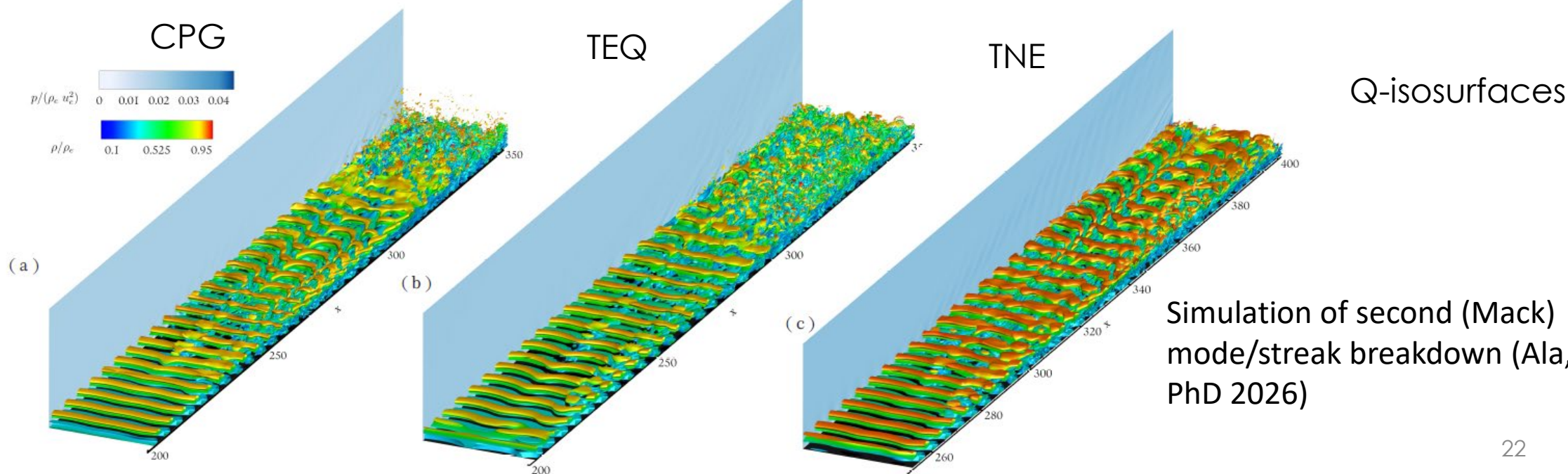
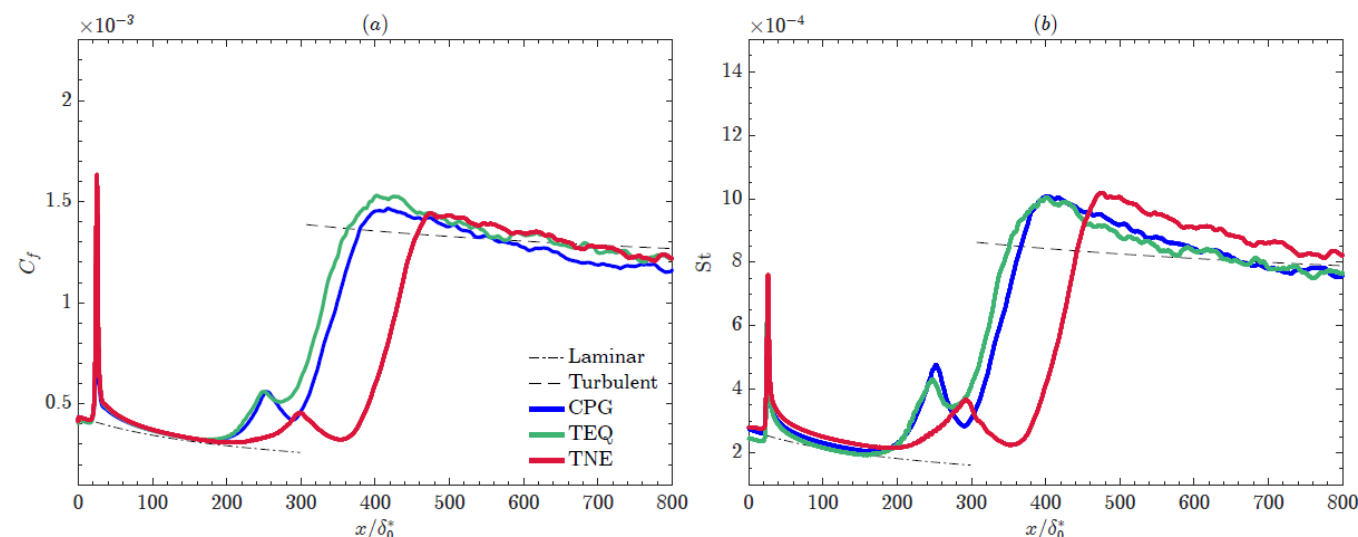


$$N_{\text{DNS}} = \ln \frac{a}{a_0} + c_{\text{recep}}$$

Transition

($M=6$ $T_w=1800K$)

- Inflow similarity solution $Re=10,000$
- 3D forcing at multiple second mode frequencies
- 5200x600x275 grid (858 million)
- Transition delay for nonequilibrium



Current interests

- Supersonic and 'cold' hypersonic regime:
 - mechanisms of transition and turbulence, transition modelling, intakes
- Leading-edge receptivity
 - Conversion of free stream disturbances into instability waves
- 'Hot' hypersonic regime: vibrational excitation and dissociation reactions
 - Mixing layers, channel flows
- Surface roughness: flow over ablated surfaces, surface chemistry, extension to slip regime
- Aero-optics